



ELEC3609/9609 - Week 3 Lab

03 Start Your First Django Web App

In this lab, you'll set up your first Django project and app, laying the groundwork for building dynamic web applications. You'll learn how Django handles routing, views, and template rendering through its structured framework.

Individual Task: Individual Task 3 - Implement Comment Features with AJAX

Lab Resource: TIL-Lab03 (Today I Learned project) source code.

Learning Objectives

After completing this lab, students should be able to:

- Set up and run a Django project and app locally.
- Understand the purpose of Django's project and app structure.
- Create views and connect them with templates.
- Configure URL routing using `urls.py`.
- Use path converters to pass parameters through URLs.

Chapter 1. Django Overview

1.1 What is Django?

Django is a Python web framework that makes the development of web applications easier with less code. It comes with a powerful set of tools out-of-the-box and follows a clean architecture pattern that encourages modular and maintainable design.

Key Features

- Django's built-in **ORM (Object-Relational Mapper)** allows you to define your data

models entirely in Python. You don't need to write raw SQL, the ORM handles table creation and maintenance automatically.

- You connect your database via a config file and manage schema changes using the `makemigrations` and `migrate` commands.
- Django follows the **MVT architecture (Model–View–Template)** to separate business logic from presentation and data access.
- URL routing is defined using declarative patterns, which helps decouple URLs from views and keeps routing flexible.

1.2 Django Architecture: Model–View–Template (MVT)

Django encourages separating an application into three components:

Component	Type	Description
Model	Purpose	Defines the structure of your database and relationships between data types. (Data)
	Responsibilities	1. Define database schema (tables, fields, relationships)
		2. Manage data access and CRUD operations
		3. Provide an API for interacting with the data
View	Purpose	Handles how data is processed and what gets displayed to the user. (Business Logic)
	Responsibilities	1. Handle user requests and responses
		2. Fetch or modify data from the Model
		3. Pass data to the Template
Template	Purpose	Presents the data to the user in HTML format. (Presentation)

Component	Type	Description
	Responsibilities	1. Render HTML using data from the View
		2. Use Django's template language to inject dynamic content
		3. Manage layout, formatting, and presentation rules

! MVT VS. MVP

You may have heard of the **MVC (Model–View–Controller)** or **MVP (Model–View–Presenter)** architectural patterns. Django follows a similar pattern but with different terminology, often referred to as **MVT (Model–View–Template)**.

In Django:

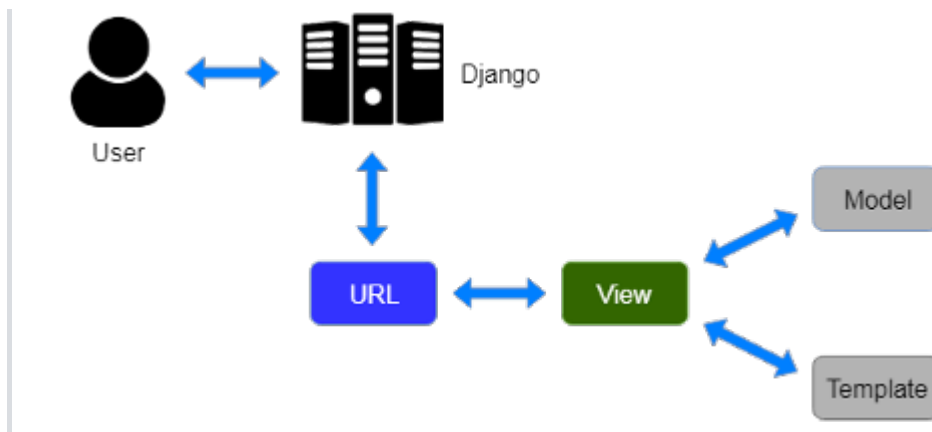
- The **View** is actually the controller logic (Python function/class handling a request).
- The **Template** is responsible for the presentation layer (what the user sees).
- The **Framework itself** handles the controller role, mapping URLs to views.

Django's naming emphasises a clear split between data (Model), logic (View), and presentation (Template). While not a strict MVP pattern, the structure loosely resembles it.

As Django puts it: "It comes down to getting stuff done." Names aside, Django offers a clean and efficient way to build web apps. Read [here](#) about how Django discusses the MVT naming.

1.3 How a Request Flows Through Django

Request → URL Dispatcher → View → Model (optional) → View → Template → Response



When a user triggers a request (e.g., by clicking a button), Django:

1. Uses the **URL dispatcher** to check if the request matches a defined route.
2. If matched, Django calls the corresponding **View function**.
3. The View may:
 - Query data from the **Model** (optional)
 - Pass this data to a **Template**
4. The Template generates an HTML response which is returned to the browser.



TIP

This pattern cleanly separates data access, business logic, and presentation. Read [here](#) about Django's design philosophies.

1.4 Built-in Security Features

Django provides several security mechanisms by default, such as:

- **CSRF tokens** to protect against cross-site request forgery
- **XSS protection** through auto-escaping in templates
- **Password hashing** with salted hashes
- Secure cookie and session handling

These features help developers build secure web applications with minimal configuration.



INFO

We'll explore Django's security features in more detail in later labs.

Chapter 2. Install Django

Being a Python web framework, Django requires Python. See [What Python version can I use with Django?](#) for details. Python includes a lightweight database called SQLite, so you won't need to set up a database just yet.

We recommend installing Django with **pip**:

```
python3 -m pip install Django
```

DJANGO AND PYTHON COMPATIBILITY

Course materials and TIL projects are tested with **Django 5.2 LTS**. When working with a provided TIL project, use its `requirements.txt` to maintain a consistent environment. If you choose to use **Django 6.0**, you must use **Python 3.12 or newer**, and some examples or third-party dependencies may behave differently.

INFO

Refer to [Quick install guide](#) for more information.

Chapter 3. Create a Django project

With Django installed, it's time to start creating our code. Django has the ability to create the stub of a project for us, so we will utilise that feature to get started.

```
django-admin startproject til_project
```

This will create the following directories and files for us.

```
til_project/  
  til_project/  
    __init__.py  
    asgi.py
```

```
settings.py
urls.py
wsgi.py
manage.py
```

These files are:

- The outer **til_project/** root directory is a container for your project. Its name does not matter to Django. You can rename it to anything you like.
- **manage.py**: A command-line utility that lets you interact with this Django project in various ways.
- The inner **til_project/** directory is the actual Python package for your project. Its name is the Python package name you'll need to use to import anything inside it.
 - **til_project/init.py**: An empty file that tells Python that this directory should be considered a Python package.
 - **til_project/settings.py**: Settings/configuration for this Django project.
 - **til_project/urls.py**: The URL declarations for this Django project.
 - **til_project/asgi.py**: An entry-point for ASGI-compatible web servers to serve your project.
 - **til_project/wsgi.py**: An entry-point for WSGI-compatible web servers to serve your project.

3.1 Development Server

Django comes with the ability to run a development server. Change into the outer **til_project** directory, and run the following commands:

```
python3 manage.py runserver
```

You will see the following output on the command line:

```
Watching for file changes with StatReloader
Performing system checks...
```

```
System check identified no issues (0 silenced).
```

```
You have 18 unapplied migration(s). Your project may not work properly until
```

```
you apply the migrations for app(s): admin, auth, contenttypes, sessions.  
Run 'python manage.py migrate' to apply them.  
August 19, 2024 - 02:10:26  
Django version 5.1, using settings 'til_project.settings'  
Starting development server at http://127.0.0.1:8000/  
Quit the server with CONTROL-C.
```

! INFO

Ignore the warning about unapplied database migrations for now, we will deal with the database in next lab.

Now that the server's running, visit **localhost** with your web browser. You will see a 'Congratulations!' page, with a rocket taking off. By default, Django binds to a local interface on port 8000. This can be configured with extra parameters to the runserver command:

```
python3 manage.py runserver 8888 # will bind to port 8888 instead of 8000  
python3 manage.py runserver 0:8888 # will bind to port 8888 on the public  
interface
```

! INFO

Generally the development server will automatically reload on code changes. Some actions, such as DB migrations, adding/deleting files, will require a restart.

3.2 settings.py

The `settings.py` file is a crucial component of a Django project. It serves as the global configuration file where you define various settings and options that control how your Django application behaves.

Setting	Purpose
<code>BASE_DIR</code>	Defines the base directory of the Django project.
<code>SECRET_KEY</code>	Used for cryptographic signing; must be kept secret in production.

Setting	Purpose
DEBUG	Toggles debug mode, providing detailed error pages during development.
ALLOWED_HOSTS	Specifies the host/domain names that the Django site can serve.
INSTALLED_APPS	Lists all the Django applications that are enabled in this project.
MIDDLEWARE	Lists middleware components that process requests/responses globally.
ROOT_URLCONF	Specifies the root URL configuration module for the project.
TEMPLATES	Configures template loading, rendering, and context processors.
DATABASES	Defines the database configurations, including engine and connection details.
STATIC_URL	Specifies the URL path for serving static files (CSS, JavaScript, images).
STATICFILES_DIRS	Lists additional directories for static files outside of app-specific locations.
LANGUAGE_CODE	Sets the default language code for the project.
TIME_ZONE	Sets the default time zone for the project.
WSGI_APPLICATION	Defines the WSGI application entry point for deployment.
ASGI_APPLICATION	Defines the ASGI application entry point for asynchronous deployment.

Chapter 4. Create a Django application

A Django project can contain multiple applications. A project contains all the components necessary for our website to function. For now, we will use a single project and application. A single application will suffice for most websites that are being built this semester, and so we don't recommend creating multiple applications since it can cause confusion and requires a richer understanding of Python/Django to correct the import path.

PROJECTS VS. APPS

A project is a collection of configuration and apps for a particular website. A project can contain multiple apps. An app can be in multiple projects.

For example, in a bank system, you might have separate apps like accounts for user authentication, transactions for handling deposits and withdrawals, and support for customer service. This modular approach allows for organised, maintainable code where each app can be developed, tested, and reused independently within the larger project.

To create a new application, we run the following command:

```
python3 manage.py startapp til_app
```

Django will create a directory structure scaffold for our TIL application, roughly similar to:

```
til_project/  
  db.sqlite3  
  manage.py  
  til_app/  
    migrations/  
      __init__.py  
    __init__.py  
    admin.py  
    apps.py  
    models.py  
    tests.py  
    views.py  
  til_project/  
    __init__.py  
    settings.py  
    urls.py  
    asgi.py  
    wsgi.py
```

til_app/ overview:

- **migrations/**: Contains migration files to apply changes to the database schema.
- **init.py**: Marks the directory as a Python package.
- **admin.py**: Registers models with the Django admin site.
- **apps.py**: Configures the Django app.
- **models.py**: Defines the data models (database schema).
- **tests.py**: Contains tests for the application.
- **views.py**: Defines the views that handle web requests and return responses.

Chapter 5. Views

In Django, a view is a function or a class that receives a web request and returns a web response. It's the core component of Django's MVC (Model-View-Controller) / MTV (Model-Template-View) architecture.

5.1 Create the First TIL View

Now that we have the basic structure of our project and application constructed, let's start making some modifications. Let's attempt to replace the default Django splash screen with our own index page.

Open the file `til_app/views.py` and create the first index view:

```
# til_app/views.py
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world. You're at the TIL index page.")
```

This is the most basic view possible in Django. To access it in a browser, we need to map it to a URL - and for this we need to define a URL configuration, or "URLconf" for short. These URL configurations are defined inside each Django app, and they are Python files named `urls.py`.

Create a file `til_app/urls.py` with the following content:

```
# til_app/urls.py
from django.urls import path

from . import views

urlpatterns = [
    path("", views.index, name="index"),
]
```

The next step is to configure the global URL conf in the **til_project** project to include the URL conf defined in **til_app.urls**. To do this, add an import for `django.urls.include` in `til_project/urls.py` and insert an `include()` in the `urlpatterns` list:

```
# til_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('til_app/', include('til_app.urls')),
]
```

! INFO

You have now wired an index view into URLconf at localhost:8000/til_app/.

If you visit localhost:8000/, you will see a “404 Page not found” error, that’s expected, as we haven’t set up a view for the root page.

5.2 URL Dispatcher

Django has the concept of URL map, which maps URL patterns in incoming requests to individual view functions. The URL dispatcher is Django’s mechanism for routing incoming HTTP requests to the appropriate view based on the URL.

Mechanism:

- **URL Matching:** When a request is received, Django starts at the root `urls.py` and sequentially checks each pattern using a top-down approach.

- **Pattern Resolution:** Each pattern is checked in order. If `path()` matches the beginning of the URL, Django passes control to the corresponding view function or includes another `urls.py` module using `include()`.
- **Capturing Parameters:** If the pattern includes variables (e.g., `<int:id>`), these are captured and passed as arguments to the view.
- **No Match:** If no pattern matches, Django returns a 404 error.

path() Function

The `path()` function is used to define URL patterns in Django. It maps a specific URL pattern to a corresponding view function or class-based view.

```
# path(route, view, kwargs=None, name=None)
path(
    route='hello/',          # URL pattern string
    view=views.index,        # Callable view function or class-based
                             # view
    kwargs=None,             # Optional dictionary of keyword
                             # arguments
    name='hello'             # Optional name for the URL pattern
)
```

include() Function

The `include()` function allows you to reference other URL configuration modules. It's typically used to include URLs from different apps, promoting a modular approach to URL management.

```
# include(module, namespace=None)
include(
    module='blog_app.urls',  # Module to include, typically a path to
                             # another app's urls.py
    namespace=None           # Optional namespace string to prevent name
                             # collisions
)
```

Path Converters

Path converters in Django are a way to capture certain parts of a URL and pass them as

arguments to the view function. They are used within the `path()` function to define and extract dynamic segments of a URL, such as integers, strings, slugs, etc.

Common Path Converter	Description
<code><str:></code>	Matches any non-empty string (this is the default converter).
<code><int:></code>	Matches an integer and passes it as an integer.
<code><slug:></code>	Matches a slug string (letters, numbers, underscores, or hyphens).
<code><uuid:></code>	Matches a universally unique identifier (UUID).
<code><path:></code>	Matches any non-empty string, including slashes (<code>/</code>).

```
urlpatterns = [
    path('post/<int:id>/', post_view, name='post_detail'), # Captures an
    # integer (Post ID) from the URL
    path('user/<str:username>/', user_profile_view),      # Captures a
    # string (Username)
]
```

Regular Expressions and `re_path()`

While `path()` is usually sufficient for most URL patterns, there are cases where you need more complex pattern matching. `re_path()` allows you define URL patterns using regular expressions for more advanced URL matching.

```
from django.urls import path, re_path
from . import views

urlpatterns = [
    path("articles/2003/", views.special_case_2003),      # 2003 only
    re_path(r"^articles/(?P<year>[0-9]{4})/$",           # yyyy
            views.year_archive),
    re_path(r"^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/$",
            views.month_archive),                        # yyyy/mm
]
```

```

    re_path(r"^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/(?P<slug>[\w-]+)/$",
            views.article_detail,
            article
        ),
    ]

```

**TIP**

Regular expressions are supported in Django via `re_path()`, but they are not commonly used in modern projects. Instead, you should design clearer and more structured URL patterns using the `path()` function with converters.

5.3 View Function VS. Class-Based View

Class-based views provide an alternative way to implement views as Python objects instead of functions. They do not replace function-based views, but have certain differences and advantages when compared to function-based views:

Aspect	Function-Based Views (FBVs)	Class-Based Views (CBVs)
Definition	A view defined as a Python function	A view defined as a Python class
Simplicity	Simple and straightforward to implement	More complex, especially for beginners
Explicitness	Logic is clear and contained within a single function	Logic may be spread across multiple methods and classes
Reusability	Limited; requires repetition for similar views	Highly reusable through inheritance and mixins
Code Organisation	Procedural style; all logic in one function	Object-oriented; logic can be organised into methods

Aspect	Function-Based Views (FBVs)	Class-Based Views (CBVs)
Built-in Support	No built-in support for common patterns	Offers built-in generic views like <code>ListView</code> , <code>DetailView</code> , etc.
Flexibility	Provides complete control over view logic	Provides structured flexibility through class inheritance
Use Case Example	Simple form handling or custom logic	Displaying lists, detail views, or handling forms using Django's generic views

```
# Function-Based View
from django.http import HttpResponse

def hello_view(request):
    return HttpResponse("Hello, World!")
```

```
# Class-Based View
from django.views import View
from django.http import HttpResponse

class HelloView(View):
    def get(self, request):
        return HttpResponse("Hello, World!")
```



BUILT-IN CLASS-BASED VIEWS

Django provides many powerful built-in class-based views, which can significantly reduce boilerplate code. You're also encouraged to write your own custom class-based views when needed.

Check [here](#) for more information.

Chapter 6. Templates

Templates in Django are a powerful way to define the structure of your HTML pages. They allow you to dynamically generate HTML content based on data passed from your views

First, create a directory called **templates** inside your Django project or app directory. You can organise templates either at the project level or within each app. In the demo, we create a directory under the app. Django will look for templates in there.

```
til_project/  
  til_app/  
    templates/
```

! INFO

Your project's `TEMPLATES` setting describe how Django will load and render templates. The default settings file configures a **DjangoTemplates** backend whose `APP_DIRS` option is set to `True`. By convention **DjangoTemplates** looks for a "templates" subdirectory in each of the `INSTALLED_APPS`.

Then add the **templates** directory to the `DIRS` option in the `TEMPLATES` setting within your `settings.py` file.

```
# settings.py  
  
import os  
from pathlib import Path  
  
BASE_DIR = Path(__file__).resolve().parent.parent  
  
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [os.path.join(BASE_DIR, 'til_app/templates')],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
],
```

```
},  
]
```

⚠ WARNING

Starting from Django 5.2, the context processor

`django.template.context_processors.debug` (as shown in the example above) is no longer included by default in newly generated projects. This change was made for better clarity and safety.

If you're interested in how Django designs its tools with security and developer experience in mind, you can [read more here](#).

Now we can create an HTML file in the **templates** directory (e.g., `templates/home.html`).

```
<!-- home.html -->  
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <title>Home Page</title>  
</head>  
<body>  
  <h1>Welcome, {{ name }}!</h1>  
  <p>Now is week {{ week }}</p>  
</body>  
</html>
```

ℹ INFO

Django offers a **templating system** for assisting with generating HTTP responses. These template files are a mix of HTML and conditional templating logic to inject data into the templates and modify the output of a page.

Now, let's create our **home** view in `til_app/views.py` and wire `urls.py` to use the template

```
# til_app/views.py  
  
from django.http import HttpResponse  
from django.template import loader
```

```
def view_home(request):
    template = loader.get_template('home.html')
    context = {"name": "ELEC3609", "week": 3}
    return HttpResponse(template.render(context, request))
```

```
# til_app/urls.py

urlpatterns = [
    # ...
    path("home", views.view_home, name="home"),
    # ...
]
```

The example loads the template called `home.html` and passes it a context. Then context is a dictionary mapping template variable names to Python objects.

Load the `localhost:8000/til_app/home`, and you should see *"Welcome, ELEC3609! Now is week 3"* as we defined the name.

USE `render()`

It's very common idiom to load a template, fill a context and return an `HttpResponse` object with the result of the rendered template. Django provides a shortcut `render()`:

```
from django.shortcuts import render

def view_home(request):
    context = {"name": "ELEC9609", "week": 3}
    return render(request, 'home.html', context)
```

6.1 CSS/JS

Up until now, we have not been setting any styling/web interaction of our own. Our pages have relied on default setup applied by browsers to different HTML elements.

TIP

Using the Developer Tools (often accessed by pressing `F12` or `Ctrl+Shift+I` on

Chrome) is one of the best ways to learn and experiment with JavaScript and CSS. It can provide you with **Immediate Feedback, Debugging, Element Inspection, Network Analysis, Mobile Emulation, Performance Monitoring**, etc.

Custom Static Files in Django

In Django, similar to templates, you can create a static directory either at the project level or within each app. In demo, we put statics within TIL app.

```
til_project/  
  til_app/  
    static/  
    templates/
```

Then, define the base URL for serving static files and list the directories where Django should look for them.

```
# settings.py  
  
STATIC_URL = '/til_app/static/' # The URL prefix for serving static files.  
STATICFILES_DIRS = [os.path.join(BASE_DIR, 'til_app/static')] # A list of  
directories where Django should look for additional static files.
```

To use static files in the templates, load the `static` template tag and reference the files.

```
{% load static %}  
<link rel="stylesheet" type="text/css" href="{% static 'css/style.css' %}">  
<script src="{% static 'js/script.js' %}"></script>
```

Inline Styles and Scripts

Use inline styles and scripts when you have simple, page-specific CSS/JS or when quick adjustments are needed without affecting other pages.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Inline CSS/JS Example</title>

<!-- Inline CSS -->
<style>
    body {
        background-color: lightblue;
    }
    h1 {
        color: white;
        text-align: center;
        margin-top: 20px;
    }
    .custom-button {
        background-color: darkblue;
        color: white;
        padding: 10px 20px;
        border: none;
        cursor: pointer;
    }
</style>
</head>
<body>
    <h1>Welcome to Django!</h1>
    <button class="custom-button" onclick="showMessage()">Click Me</button>

    <!-- Inline JavaScript -->
    <script>
        function showMessage() {
            alert("Hello, Django!");
        }
    </script>
</body>
</html>
```

External Libraries

You can include external CSS and Javascript libraries like Bootstrap, jQuery by linking to them via a CDN (Content Delivery Network). External libraries are ideal for quickly adding well-tested, feature-rich CSS and Javascript functionality to your application without managing the library files yourself.

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>External CSS/JS Example</title>

<!-- External CSS (Bootstrap) -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/
bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <div class="container text-center">
    <h1 class="mt-5">Welcome to Django with Bootstrap!</h1>
    <button class="btn btn-primary mt-3" id="bootstrapButton">Click Me</
button>
  </div>

  <!-- External JavaScript (Bootstrap and jQuery) -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/
bootstrap.bundle.min.js"></script>

  <!-- Custom JavaScript using jQuery -->
  <script>
    $(document).ready(function(){
      $('#bootstrapButton').click(function(){
        alert("Hello, Django with Bootstrap and jQuery!");
      });
    });
  </script>
</body>
</html>
```

**TIP**

We've added some basic HTML, CSS, and JavaScript examples under `til_app/templates` and `til_app/static`. Have a look if you're new to HTML or front-end development.

6.2 AJAX

AJAX stands for Asynchronous JavaScript and XML. AJAX enables you to provide a seamless website experience for your users without the interruption of reloads. AJAX is a technique used in web development to make web pages more interactive and responsive without

needing to reload the entire page every time a user interacts with it.

! INFO

Watch the [following video](#) to get a better understanding of AJAX.

Clever Techie



Watch on

6.3 Modern Frontend

While Django provides a powerful server-side templating system, many modern web applications now rely on frontend libraries and frameworks like React to create more dynamic, interactive, and responsive user interfaces. These tools are widely used in today's web development to build fast, efficient, and maintainable applications.

Technology	Description
TypeScript	A superset of JavaScript that adds static typing to the language, helping developers catch errors early and making code more predictable and easier to refactor.
React	A JavaScript library for building dynamic and interactive user interfaces, particularly single-page applications, using reusable components.
Angular	A full-fledged JavaScript framework developed by Google for building large-scale, complex web applications with a focus on modularity and

Technology	Description
	maintainability.
Vue.js	A progressive JavaScript framework that is easy to integrate and is used for building user interfaces with a strong focus on simplicity and reactivity.
jQuery	A fast, small, and feature-rich JavaScript library that simplifies HTML DOM manipulation, event handling, and AJAX interactions, although it's less commonly used in new projects today due to modern alternatives like React and Vue.
Bootstrap	A popular CSS framework that provides pre-designed components and a responsive grid system to quickly build mobile-first and responsive websites.
Tailwind CSS	A utility-first CSS framework that allows developers to create custom designs directly in their HTML using pre-defined utility classes.
Vite	A build tool and development server that provides a fast and efficient workflow for modern web development, optimized for frameworks like React, Vue, and others.
Redux	A state management library for JavaScript apps, often used with React, to manage the global state of an application in a predictable and centralized manner.
Axios	A promise-based HTTP client for making requests to APIs, often used in JavaScript and React applications for fetching and posting data.

Chapter 7. More Functions

7.1 Create "*View a Post*"

To keep things simple, we demonstrate viewing a post directly defining a `Post` class and storing a few sample posts in memory in `til_app/views.py`:

```
# til_app/views.py
class Post:
    def __init__(self, post_id, author, title, content=None, img=None,
comments=None):
        self.post_id = post_id
        self.author = author
        self.title = title
        self.content = content
        self.img = img
        self.comments = comments
```

```
post1 = Post(1, 'Jwen', 'First Post', content='Cat', img='https://
```

```
# til_app/views.py
def view_post(request, post_id):
    post = next((p for p in posts if p.post_id == post_id), None)

    if post:
        return render(request, 'post/post.html', {'post': post})

    return render(request, 'post/post.html', {'error': 'Post not found'},
status=404)
```

This renders the post using the template at `til_app/templates/post/post.html`:

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{{ post.title }}</title>
</head>
<body>
    <h1>{{ post.title }}</h1>
    <p>by {{ post.author }}</p>
    <p>{{ post.content }}</p>
    

    <h2>Comments</h2>
    <ul id="commentList">
        {% for comment in post.comments %}
            <li>{{ comment }}</li>
```

```

        {% endfor %}
    </ul>

    <h3>Upload a Comment</h3>
    <form id="commentForm">
        {% csrf_token %}
        <textarea name="comment" required></textarea><br>
        <button type="button" onclick="submitComment({{ post.post_id
    }})">Add Comment</button>
    </form>

    <script src="{% static 'script.js' %}"></script>
</body>
</html>

```

**TIP**

You'll notice we use `{{ post.title }}`, `{{ post.author }}`, etc., these are part of Django's template language, where variables passed from the view function are rendered using double curly braces. This is how Django dynamically fills in HTML content based on your data.

Finally, we wire the route using a **path converter** in `til_app/urls.py`:

```

# til_app/urls.py
urlpatterns = [
    path("post/<int:post_id>", views.view_post, name="view a post"),
    # ...
]

```

So now if you visit `localhost:8000/til_app/post/1`, you'll see the first post rendered dynamically.

7.2 Create "View Posts"

To display a list of all posts, we first create a template at `til_app/templates/post/posts.html`:

```

{% load static %}
<!DOCTYPE html>

```

```

<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Posts</title>
  <link rel="stylesheet" href="{% static 'style.css' %}">
</head>

<body>
  <h1>All Posts</h1>
  <ul id="postList">
    {% for post in posts %}
      <li>
        <a href="{% url 'view a post' post.post_id %}">
          <strong>{{ post.title }}</strong> by {{ post.author }}
        </a>
        <br>
        
      </li>
    {% endfor %}
  </ul>

  <script src="{% static 'script.js' %}"></script>
</body>

</html>

```

In this template, we introduce **Django's template tags** using `{% %}`:

- `{% load static %}` loads Django's static file tag system so we can link to CSS/JS.
- `{% for post in posts %}` loops through the list of posts passed from the view.
- `{% url 'view a post' post.post_id %}` dynamically generates the URL to view a single post based on its ID.

! INFO

`{% %}` is used for logic and control structures in templates, like loops and loading static files. This is different from `{{ }}`, which is used to output variables like `{{ post.title }}` or `{{ post.author }}`.

Next, we define a simple view function in `til_app/views.py`:

```
# til_app/views.py
def view_posts(request):
    context = {"posts": posts}
    return render(request, 'post/posts.html', context)
```

And wire it to a URL in `til_app/urls.py`:

```
# til_app/urls.py
urlpatterns = [
    path("posts", views.view_posts, name="view posts"),
    # ...
]
```

Now, visiting `localhost:8000/til_app/posts` will show you a list of posts with titles, authors, and thumbnail images, each linking to their detailed view.

7.3 Create "Upload a Post"

To implement a function with AJAX, we start with an HTML form that the user fills out. This form will collect data like the author's name, the post title, and the content of the post. In `til_app/templates/post/posts.html`, we add the following form:

```
<form id="uploadForm">
    {% csrf_token %}
    <label for="author">Author:</label>
    <input type="text" id="author" name="author"><br>

    <label for="title">Title:</label>
    <input type="text" id="title" name="title"><br>

    <label for="content">Content:</label>
    <textarea id="content" name="content"></textarea><br>

    <label for="img">Image URL:</label>
    <input type="text" id="img" name="img"><br>

    <button type="submit">Submit Post</button>
```

```
</form>
```

```
<script src="{% static 'script.js' %}"></script>
```

⚠️ CSRF TOKEN

The `{% csrf_token %}` tag ensures that the form submission is secure by including a CSRF token. This token must be included in AJAX requests to prevent cross-site request forgery.

Then, JavaScript works to submit the form via AJAX. In `til_app/static/script.js`:

```
// AJAX for 'Upload a Post'
document.getElementById('uploadForm').addEventListener('submit',
function(event) {
    event.preventDefault(); // Prevent the default form submission (page
    reload)

    let formData = new FormData(this); // Create a FormData from user input

    // Send the form data using the Fetch API
    fetch('/til_app/posts/upload', {
        method: 'POST',
        body: formData,
        headers: {
            'X-CSRFToken': formData.get('csrfmiddlewaretoken') // Include
the CSRF token in the headers
        },
    })
    .then(response => response.json()) // Parse the JSON response from the
server
    .then(data => {
        if (data.custom_error_code === 0) {
            // Instead of reloading the page, dynamically add the new post
to the DOM
            addPostToDOM(data);
            document.getElementById('uploadForm').reset(); // Clear the
form fields
        } else if (data.error) {
            alert(data.error); // Display any error messages
        }
    })
    .catch(error => console.error('Error:', error)); // Handle any errors
```

```
in the request
});
```

On the Django side, we need a view that handles this POST request, processes the form data, and returns a JSON response. In `til_app/views.py`, we add `create_post` method:

```
# Upload a Post
def upload_post(request):
    if request.method == 'POST':
        author = request.POST.get('author')
        title = request.POST.get('title')
        content = request.POST.get('content')
        img = request.POST.get('img')

        # Bad Design !!!
        post_id = len(posts) + 1
        new_post = Post(post_id, author, title, content, img)
        posts.append(new_post)

        # Sends a JSON response back to the client.
        # If the post is successfully created, return any new post info for
        rendering.
        return JsonResponse({
            "custom_error_code": 0, # custom status code
            "post_id": post_id,
            "author": author,
            "title": title,
            "img": img
        }, status=200)

    return JsonResponse({'error': 'Invalid request'}, status=400)
```

And finally, we wire the view to URL dispatcher in `til_app/urls.py`:

```
# til_app/urls.py
urlpatterns = [
    # ...
    path("posts/upload", views.upload_post, name="upload a post"),
]
```

How AJAX Works in This Context

- **Asynchronous:** The form submission is handled asynchronously, meaning the page doesn't reload. The user can continue interacting with the page while the data is being processed.
- **Data Exchange:** Data is sent to the server (in this case, the post data) and the server processes it, storing the post and sending back a response indicating success or failure.
- **User Experience:** The page remains responsive, and the user receives immediate feedback (e.g., the new post appears on the page or an error message is shown).

Access to localhost:8000/til_app/posts to see the behaviour.

Chapter 8. Development Setup for Team

If you're not familiar with how to set up a Python development environment for a team project, read the following notes about environment setup for a Python project.

▶ Environment Setup for a Python Project (Optional Reading)

Further Readings

- [Writing your first Django app, part 1](#)
- [django.urls](#) functions for use in URLconfs
- [Django URL dispatcher](#)
- [Class-based views](#)
- [Django Templates](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from

prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.